Estrategia de Pruebas – CineFlex

Fase A: Arquitectura y Estrategia de Pruebas · Taller Práctico Integrador: QA en Producción

0. Alcance y arquitectura bajo prueba

CineFlex es una plataforma web de venta de boletos de cine compuesta por tres capas: un frontend (SPA en HTML/JS con vistas de login, cartelera, asientos, pago y ticket), un backend REST en Node.js/Express protegido con Helmet, trazado con Morgan y autenticado con JWT, y una capa de datos/terceros donde vive la pasarela de pagos externa del banco. El objetivo es certificar que la plataforma soporte la noche del gran estreno mundial sin fallas funcionales ni de rendimiento.

Capa	Componente	Tipo de prueba	Herramienta
Frontend	frontend/index.html	Sistema / E2E / Regresión UI	Selenium WebDriver + POM
Backend	backend/server.js (/api/login, /api/cartelera, /api/asientos/:id, /api/reservas)	API / Integración / Seguridad	Postman + Newman
Terceros	Pasarela bancaria (simulada con Stub)	Integración	Stub en Express
Infraestructura	API bajo concurrencia	Carga y estrés	Apache JMeter
Pipeline	GitHub Actions	Continuous Testing	Newman en cada push

1. Pruebas de Integración: Stubs y Drivers

1.1 Enfoque elegido: Top-Down

En CineFlex ya están terminados los módulos superiores (la interfaz de pagos en el frontend y el endpoint **POST /api/reservas** en el backend). El único módulo que falta es el de más bajo nivel: la API del banco. Por eso el enfoque correcto es **Top-Down**: se integra desde la UI hacia abajo y el componente ausente se reemplaza por un **Stub**. En Bottom-Up ocurriría lo contrario: tendríamos el módulo bajo terminado y necesitaríamos un **Driver** (un programa que lo invoque) porque faltaría la capa superior; esa no es nuestra situación.

Analogía: el Stub es el doble de acción de una película. El actor principal (el banco) todavía no llegó al set, pero el doble responde en su lugar con movimientos predecibles para que el resto del rodaje (CineFlex) no se detenga.

1.2 Diseño del Stub de la pasarela bancaria

- **Contrato:** recibe un objeto *tarjeta* {titular, numero, vencimiento, cvv} y devuelve {aprobada, codigoAutorizacion} o {aprobada:false, motivo}, imitando la respuesta que documentaría el banco.
- **Datos de prueba deterministas:** las tarjetas 4111111111111111 (Visa) y 5555555555554444 (Mastercard) se aprueban; cualquier otro número se rechaza. Así cada caso de prueba tiene un resultado esperado conocido.

- **Punto único de reemplazo:** el Stub vive en una sola función de *server.js*. Cuando el banco entregue su sandbox, solo se sustituye esa función por la llamada HTTP real, sin tocar el endpoint ni el frontend.
- **Mapeo a HTTP:** pago aprobado → 201 Created con ticket; pago rechazado → 402 Payment Required con el mensaje "Pago rechazado: tarjeta inválida"; datos incompletos → 400.

```
// backend/server.js
function pasarelaBancariaStub(tarjeta) {
  const TARJETAS_APROBADAS = ['4111111111111111', '5555555555554444'];
  if (tarjeta && TARJETAS_APROBADAS.includes(tarjeta.numero)) {
    return { aprobada: true, codigoAutorizacion: 'AUTH-' + Date.now() };
  }
  return { aprobada: false, motivo: 'Tarjeta rechazada por el banco' };
}

// Dentro de POST /api/reservas
const pago = pasarelaBancariaStub(tarjeta);
if (!pago.aprobada) return res.status(402).json({ error: 'Pago rechazado: tarjeta inválida' });
```

1.3 Cómo se prueba hoy mismo

- **Postman/Newman:** el request "Reserva con tarjeta falsa" verifica 402, el mensaje exacto y que no se emita ticket.
- **Selenium:** CP12 valida el rechazo desde la UI y CP13 completa la compra End-to-End con la tarjeta aprobada, verificando que aparezca el ticket virtual.
- **Riesgo residual:** el Stub no prueba latencia, timeouts ni códigos reales del banco. Se documenta como pendiente para la fase de integración con el sandbox real (prueba de contrato).

2. Pruebas de Sistema y Regresión

2.1 Requisitos

ID	Tipo	Requisito	Cómo se verifica
RF-01	Funcional	El usuario debe autenticarse con email y contraseña; tras 3 intentos fallidos consecutivos el sistema bloquea el inicio de sesión.	Selenium CP02–CP05 · Postman Login
RF-02	Funcional	El usuario autenticado debe poder seleccionar una película, elegir un asiento libre, pagar y recibir un ticket virtual.	Selenium CP09–CP13 · Postman Reserva
RNF-01	Rendimiento	La consulta de cartelera (GET /api/cartelera) debe responder en menos de 300 ms.	Postman: pm.response.responseTime < 300
RNF-02	Escalabilidad / Seguridad	La consulta de asientos debe soportar 500 usuarios concurrentes (ramp-up 15 s) con error < 1 %; toda respuesta debe incluir cabeceras de seguridad (Helmet).	JMeter Summary Report · Postman header X-Content-Type-Options

2.2 Estrategia de Regresión: selectiva basada en riesgo e impacto

Caso planteado: un desarrollador cambia el color de un botón secundario en la vista del perfil. **No** volveríamos a ejecutar toda la suite de la pasarela de pagos. El cambio es puramente visual (CSS), no toca la lógica de */api/reservas*, el Stub ni el formulario de pago, por lo que el riesgo sobre pagos es prácticamente nulo. Ejecutar toda la suite de pagos consumiría minutos de CI y tiempo del equipo sin aportar información nueva.

Criterio de selección de casos:

Tipo de cambio	Suite que se ejecuta
Solo estilos/CSS de una vista no crítica (ej. botón del perfil)	Pruebas UI de esa vista + Smoke test (CP01, CP05, CP13)
Lógica de login o JWT	Suite completa de Login (CP01–CP05, CP14–CP16) + Postman Login y Cartelera
Endpoint /api/reservas, Stub bancario o formulario de pago	Suite completa de pagos (CP09–CP13) + Postman Reserva + JMeter
Dependencias, Express, Helmet o release a producción	Regresión total: Newman + 16 casos Selenium + JMeter

En el pipeline, Newman corre en cada *push* porque es rápido (menos de 1 s) y protege el núcleo del negocio. La suite Selenium completa y JMeter se reservan para merges a *main* y releases, optimizando costos de ejecución.

3. Comparativa de Herramientas: Selenium WebDriver vs Cypress

Criterio	Selenium WebDriver	Cypress
Escenario CineFlex recomendado	**Negocio:** certificar la compra de boletos en Chrome, Firefox, Edge y Safari antes del estreno, porque los clientes llegan desde cualquier navegador y dispositivo.	**Técnico:** pruebas rápidas del frontend durante el desarrollo diario (mapa de asientos, validaciones del formulario de pago) con recarga en vivo.
Navegadores	Todos, incluido Safari y navegadores reales en grid (Selenium Grid / nubes de dispositivos).	Chromium, Firefox y WebKit (experimental). Sin Safari real.
Lenguajes	JavaScript, Java, Python, C#, Ruby.	Solo JavaScript / TypeScript.
Arquitectura	Controla el navegador desde fuera mediante el protocolo W3C WebDriver.	Se ejecuta dentro del navegador, junto a la app; esperas automáticas.
Múltiples pestañas / dominios	Soportado: útil si la pasarela real redirige a la página del banco (3-D Secure).	Limitado (cy.origin); no maneja varias pestañas.
Velocidad y depuración	Más lento; requiere esperas explícitas (driver.wait).	Muy rápido; time-travel debugging, capturas y video automáticos.
Curva de aprendizaje	Media-alta; se compensa con el patrón POM.	Baja; API declarativa y documentación muy guiada.

Criterio	Selenium WebDriver	Cypress
Decisión para CineFlex	Suite de regresión oficial (16 casos en ui_tests/).	Complemento opcional para pruebas de componentes del equipo frontend.

4. Criterios de salida (listo para producción)

- 100 % de aserciones Newman en verde en el pipeline (bloqueo de merge si falla).
- 16/16 casos Selenium aprobados, incluido el flujo E2E y la navegación segura.
- JMeter con 500 usuarios: error < 1 % y throughput documentado en el README.
- Cabeceras de seguridad Helmet presentes y trazabilidad Morgan activa en todas las peticiones.